

✓ Proyecto: Big Data

Evaluación del módulo Introducción al procesamiento distribuido y sistemas Big Data

Objetivo del Proyecto

Diseñar e implementar un **pipeline de Big Data con Apache Spark** que:

- Lea datos en formatos JSON, CSV y Parquet.
 - Procese datos estructurados y no estructurados.
 - Soporte procesamiento batch y en streaming.
 - Entrene un modelo de aprendizaje automático escalable con MLlib.
-

Lección 1: Big Data

Tareas:

1. Definir las 5V's del Big Data aplicadas al caso:

- **Volumen:** Datos masivos de ventas, IoT, redes sociales y logs.
- **Velocidad:** Requiere procesamiento en tiempo real (streaming).
- **Variedad:** Datos estructurados (ventas) y no estructurados (logs, redes sociales).
- **Veracidad:** Es necesario limpiar datos ruidosos de sensores/redes.
- **Valor:** Se busca extraer insights para la toma de decisiones.

2. Beneficios del enfoque distribuido vs. local:

- Escalabilidad horizontal.
- Tolerancia a fallos.
- Mayor velocidad al procesar datos en paralelo.

3. Mapa de tecnologías clave:

- Apache Spark (Core, SQL, Streaming, MLlib).
 - HDFS o almacenamiento distribuido (simulado).
 - Kafka o sockets (entrada en tiempo real).
 - JSON, CSV, Parquet (formatos).
-

Lección 2: Apache Spark

Tareas:

1. ¿Cuándo usar Spark?

- Cuando se necesita procesar datos a gran escala, en memoria y con baja latencia.
- Cuando se combinan batch, streaming, ML y SQL.

2. Arquitectura de Spark:

- **Driver:** Programa principal, coordina todo.
- **Executors:** Ejecutan tareas.
- **Cluster Manager:** Asigna recursos (YARN, Kubernetes, etc.).

3. Módulos a usar en el proyecto:

- Spark Core
 - Spark SQL
 - Spark Streaming
 - MLlib
-

Lección 3: Procesamiento distribuido con RDDs

Tareas:

1. Crear entorno local en Colab o Databricks.
2. Usar `parallelize()` y `textFile()` para cargar datos.
3. Aplicar transformaciones (`map`, `filter`, `sortBy`, etc.).
4. Aplicar acciones (`count`, `collect`, `mean`).

- Documentar el flujo de ejecución y el Job Spark.

Lección 4: Datos estructurados y Spark SQL

Tareas:

- Cargar datos JSON, CSV, Parquet.
- Realizar consultas SQL.
- Usar UDFs (funciones definidas por el usuario).
- Comparar rendimiento vs RDD.

Lección 5: Procesamiento en streaming

Tareas:

- Leer datos en tiempo real desde socket o archivo simulado.
- Aplicar transformaciones sobre DStreams.
- Implementar ventana temporal (`window`, `reduceByKeyAndWindow`).
- Comparar latencia con procesamiento batch.

Lección 6: ML Escalable con MLlib

Tareas:

- Preparar datos: encoding, escalado, selección de features.
- Entrenar modelo supervisado (ej. clasificación con `LogisticRegression`).
- Evaluar desempeño (`AUC`, `accuracy`, `RMSE`, etc.).
- Guardar modelo (`.save()`).
- Preparar predicción sobre datos batch o streaming.

✓ Notebooks/Códigos

```
1 from pyspark.sql import SparkSession
2
3 spark = SparkSession.builder \
4     .appName("Colab Spark Default") \
5     .getOrCreate()
6
7 df = spark.createDataFrame([(1, "ok"), (2, "sin descargas")], ["id", "estado"])
8 df.show()
9
```

```
+---+-----+
| id|      estado|
+---+-----+
|  1|          ok|
|  2|sin descargas|
+---+-----+
```

✓ notebook 01 Ingesta Batch Spark

```
1 # -----
2 # 1. Leer archivo CSV: ventas.csv
3 ventas = spark.read.option("header", True).option("inferSchema", True).csv("/content/sample_data/ventas.csv")
4
5 print("Ventas:")
6 ventas.show(truncate=False)
7 ventas.printSchema()
8
9 # -----
10 # 2. Leer archivo JSON: sensores.json
11 sensores = spark.read.option("multiline", True).json("/content/sample_data/sensores.json")
12
13 print("Sensores:")
14 sensores.show(truncate=False)
15 sensores.printSchema()
16
17 # Convertimos timestamp a formato adecuado
18 from pyspark.sql.functions import to_timestamp
19
```

```

20 sensores = sensores.withColumn("timestamp", to_timestamp("timestamp"))
21
22 # -----
23 # 3. Leer archivo de texto: logs.txt (formato no estructurado)
24 import re
25
26 logs_rdd = spark.sparkContext.textFile("/content/sample_data/logs.txt")
27
28 # Expresión regular para extraer [timestamp] - [nivel] - [mensaje]
29 pattern = r"(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) - (\w+) - (.*)"
30
31 logs_df = logs_rdd \
32     .map(lambda line: re.findall(pattern, line)) \
33     .filter(lambda x: len(x) > 0) \
34     .map(lambda x: x[0]) \
35     .toDF(["timestamp", "nivel", "mensaje"]) \
36     .withColumn("timestamp", to_timestamp("timestamp"))
37
38 print("Logs:")
39 logs_df.show(truncate=False)
40 logs_df.printSchema()
41
42 # -----
43 # 4. Guardar en disco
44 ventas.write.mode("overwrite").parquet("/content/sample_data/output/ventas_parquet")
45 sensores.write.mode("overwrite").json("/content/sample_data/output/sensores_json")
46 logs_df.write.mode("overwrite").parquet("/content/sample_data/output/logs_parquet")
47
48 # -----
49 # 5. Validar lectura post-exportación
50 print("✅ Archivos exportados. Verificación:")
51 spark.read.parquet("/content/sample_data/output/ventas_parquet").show(3)
52 spark.read.json("/content/sample_data/output/sensores_json").show(3)
53 spark.read.parquet("/content/sample_data/output/logs_parquet").show(3)
54

```

Ventas:

	id_venta	fecha	cliente	producto	categoria	cantidad	precio_unitario
1		2025-06-03 04:04:08	Sara Jacobs	now	Hogar	5	8.77
2		2025-02-23 05:00:49	Karen Wright	shake	Ropa	5	189.44
3		2025-03-12 09:33:42	Elizabeth Warner	little	Electrónica	2	52.83
4		2025-05-04 07:43:57	Emily Smith	that	Hogar	5	335.19
5		2025-01-17 00:05:13	David Wood	certainly	Hogar	1	160.1
6		2025-02-20 14:10:00	Mary Turner	myself	Libros	2	92.01
7		2025-05-23 17:05:45	Dawn Wilson	audience	Ropa	5	87.03
8		2025-08-22 07:04:12	Kyle Fischer	about	Hogar	4	450.33
9		2025-02-10 02:52:47	Andrew Douglas	college	Electrónica	4	203.28
10		2025-09-01 12:09:53	Frederick Byrd	bag	Libros	5	152.23
11		2025-02-22 05:24:02	David Clark	modern	Hogar	5	215.5
12		2025-05-23 01:33:52	Steven Pope	individual	Ropa	5	285.32
13		2025-08-06 01:02:41	Jenna Robinson	both	Ropa	3	439.75
14		2025-04-28 08:26:11	Robert Morrison	just	Electrónica	5	454.59
15		2025-08-22 13:38:33	Michael Kim	pretty	Hogar	4	195.57
16		2025-02-23 08:03:46	Dr. Jessica Golden	phone	Libros	1	489.14
17		2025-03-24 04:47:57	Richard Perez	mind	Libros	4	260.22
18		2025-07-08 23:45:57	Jacob Dudley	get	Libros	1	263.92
19		2025-03-18 08:23:36	Erin Richardson	information	Electrónica	1	91.93
20		2025-05-01 02:59:19	Kathryn Jones	summer	Hogar	4	58.93

only showing top 20 rows

root

```

|-- id_venta: integer (nullable = true)
|-- fecha: timestamp (nullable = true)
|-- cliente: string (nullable = true)
|-- producto: string (nullable = true)
|-- categoria: string (nullable = true)
|-- cantidad: integer (nullable = true)
|-- precio_unitario: double (nullable = true)

```

Sensores:

```

+-----+-----+-----+-----+
|humedad|sensor_id|temperatura|timestamp|
+-----+-----+-----+-----+
|83.3   |57       |31.3      |2025-06-30T10:42:18|
+-----+-----+-----+-----+

```

root

```

|-- humedad: double (nullable = true)
|-- sensor_id: string (nullable = true)
|-- temperatura: double (nullable = true)
|-- timestamp: string (nullable = true)

```

Logs:

```

+-----+-----+-----+
|timestamp|nivel|mensaje|
+-----+-----+-----+

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|2025-06-29 17:55:13|INFO |Order dog nor with debate pass company practice find long.
|2025-02-17 19:34:05|ERROR|Unit recently practice where human while nor camera hit.
|2025-04-04 18:17:49|ERROR|Amount possible truth fine fast court Republican time alone light deep.
|2025-08-12 05:16:48|ERROR|Create story center almost rather lot.
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Notebook 02: Consultas SQL con Spark SQL

```

1 # Notebook 02: Consultas SQL con Spark
2
3 # Requiere que hayas corrido el Notebook 01 primero.
4
5 from pyspark.sql import SparkSession
6 from pyspark.sql.functions import col, lower, upper, length
7
8 # Crear la SparkSession
9 spark = SparkSession.builder.appName("Consultas SQL con Spark").getOrCreate()
10
11 # Cargar los datos desde archivos transformados
12 ventas_df = spark.read.option("inferSchema", True).csv("sample_data/ventas.csv")
13 sensores_df = spark.read.option("inferSchema", True).json("sample_data/sensores.json")
14 logs_rdd = spark.sparkContext.textFile("sample_data/logs.txt")
15
16
17 # Transformar logs RDD a DataFrame
18 import re
19 logs_df = logs_rdd.map(lambda line: re.findall(r"(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) - (\w+) - (.+)", line)) \
20     .filter(lambda x: len(x) > 0) \
21     .map(lambda x: x[0]) \
22     .toDF(["timestamp", "nivel", "mensaje"])
23
24 # Cambiar tipo de timestamp (string a timestamp real)
25 from pyspark.sql.functions import to_timestamp
26 logs_df = logs_df.withColumn("timestamp", to_timestamp("timestamp"))
27
28 # Registrar tablas temporales
29 ventas_df.createOrReplaceTempView("ventas")
30 sensores_df.createOrReplaceTempView("sensores")
31 logs_df.createOrReplaceTempView("logs")
32
33 # -----
34 # Consultas SQL
35 # -----
36
37 # 1. Total de ventas por categoría
38 q1 = spark.sql("""
39     SELECT categoria, SUM(cantidad * precio_unitario) AS total_ventas
40     FROM ventas
41     GROUP BY categoria
42     ORDER BY total_ventas DESC
43 """)
44
45 # 2. Temperaturas mayores a 30 grados
46 q2 = spark.sql("""
47     SELECT sensor_id, temperatura, humedad, timestamp
48     FROM sensores
49     WHERE temperatura > 30
50     ORDER BY temperatura DESC
51 """)
52
53 # 3. Conteo de logs por nivel de mensaje (INFO, WARN, ERROR)
54 q3 = spark.sql("""
55     SELECT nivel, COUNT(*) AS cantidad
56     FROM logs
57     GROUP BY nivel
58     ORDER BY cantidad DESC
59 """)
60
61 # -----
62 # Mostrar resultados
63 # -----
64
65 print("\n Total de ventas por categoría:")
66 q1.show()
67
68 print("\n Temperaturas altas:")
69 q2.show()
70
71 print("\n Logs por nivel:")
72 q3.show()

```

```

Total de ventas por categoría:
+-----+-----+
| categoria| total_ventas|
+-----+-----+
| Libros| 19170.75|
| Hogar| 18650.410000000003|
| Electrónica| 15795.36|
| Ropa| 15110.209999999997|
+-----+-----+

```

```

Temperaturas altas:
+-----+-----+-----+-----+
| sensor_id| temperatura| humedad| timestamp|
+-----+-----+-----+-----+
| S10| 35.0| 64.3| 2025-06-03T02:54:55|
| S2| 34.8| 53.4| 2025-04-04T05:04:06|
| S2| 34.7| 45.4| 2025-04-27T07:38:54|
| S2| 34.2| 49.1| 2025-02-16T22:27:25|
| S10| 34.2| 83.5| 2025-07-01T04:49:38|
| S4| 34.1| 62.6| 2025-02-26T11:19:37|
| S1| 34.0| 46.1| 2025-09-10T04:23:17|
| S4| 34.0| 39.2| 2025-01-18T22:38:49|
| S3| 33.9| 40.7| 2025-03-18T18:11:47|
| S6| 33.7| 58.5| 2025-09-11T08:19:21|
| S4| 33.5| 73.4| 2025-06-26T10:42:57|
| S1| 33.4| 57.5| 2025-07-05T00:51:37|
| S3| 33.2| 59.2| 2025-06-11T08:41:03|
| S9| 33.2| 73.8| 2025-04-26T15:04:54|
| S5| 32.8| 36.1| 2025-08-27T00:08:21|
| S2| 32.7| 86.0| 2025-02-19T12:05:11|
| S5| 32.5| 59.6| 2025-05-26T15:39:14|
| S3| 32.5| 77.9| 2025-02-17T23:26:16|
| S2| 32.4| 72.2| 2025-07-17T11:05:42|
| S4| 32.4| 89.7| 2025-08-29T13:04:15|
+-----+-----+-----+-----+

```

only showing top 20 rows

```

Logs por nivel:
+-----+-----+
| nivel| cantidad|
+-----+-----+
| ERROR| 48|
| INFO| 40|
| WARN| 32|
+-----+-----+

```

✓ Notebook 03: Streaming con Spark

```

1 # Requiere tener una SparkSession ya creada:
2 from pyspark.sql import SparkSession
3 from pyspark.sql.functions import split, explode, current_timestamp, window
4 import os
5
6 spark = SparkSession.builder.getOrCreate()
7
8 # Paso 1: Crear archivo simulado
9 stream_file_path = "/content/streaming.txt"
10 with open(stream_file_path, "w") as f:
11     f.write("Hola mundo esto es una simulación de streaming\n")
12     f.write("Otra línea con más palabras para procesar\n")
13     f.write("Y una tercera línea para cerrar\n")
14
15 # Paso 2: Leer como archivo de texto (simulación de flujo)
16 lineas = spark.read.text(stream_file_path)
17
18 # Paso 3: Añadir marca de tiempo a cada línea como si fuese en vivo
19 con_timestamp = lineas.withColumn("timestamp", current_timestamp())
20
21 # Paso 4: Separar en palabras
22 palabras = con_timestamp.select(
23     explode(split("value", " ")).alias("palabra"),
24     "timestamp"
25 )
26
27 # Paso 5: Aplicar ventana de 10 minutos con desplazamiento de 5 min + watermark
28 conteo = palabras \
29     .withWatermark("timestamp", "5 minutes") \
30     .groupBy(window("timestamp", "10 minutes", "5 minutes"), "palabra") \
31     .count()
32
33 # Paso 6: Mostrar resultado (como si fuese output de writeStream)
34 print("✅ Conteo por ventana:")

```

```
35 conteo.orderBy("window").show(truncate=False)
36
```

✓ Conteo por ventana:

```
+-----+-----+-----+
|window|palabra|count|
+-----+-----+-----+
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|es|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|streaming|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|con|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|una|2|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|palabras|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|Hola|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|esto|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|tercera|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|de|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|más|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|línea|2|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|simulación|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|procesar|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|cerrar|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|para|2|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|mundo|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|Otra|1|
|{2025-09-17 08:15:00, 2025-09-17 08:25:00}|Y|1|
|{2025-09-17 08:20:00, 2025-09-17 08:30:00}|palabras|1|
|{2025-09-17 08:20:00, 2025-09-17 08:30:00}|línea|2|
+-----+-----+-----+
```

only showing top 20 rows

✓ Notebook 4: Machine Learning con Spark MLlib

```
1 # ✓ Notebook 04: Machine Learning con Spark MLlib
2 # Objetivo: Clasificar productos de ventas.csv según su categoría usando MLlib
3
4 from pyspark.sql import SparkSession
5 from pyspark.ml.feature import StringIndexer, VectorAssembler
6 from pyspark.ml.classification import RandomForestClassifier
7 from pyspark.ml.evaluation import MulticlassClassificationEvaluator
8
9 # Crear la sesión de Spark
10 spark = SparkSession.builder.appName("MLlib Ventas").getOrCreate()
11
12 # Cargar el dataset de ventas
13 ventas_df = spark.read.option("header", True).option("inferSchema", True).csv("/content/sample_data/ventas.csv")
14 ventas_df.show(5)
15 ventas_df.printSchema()
16
17 # Indexar la variable objetivo (categoría)
18 indexer = StringIndexer(inputCol="categoría", outputCol="categoría_index")
19 ventas_df = indexer.fit(ventas_df).transform(ventas_df)
20
21 # Crear vector de características
22 features_cols = ["cantidad", "precio_unitario"]
23 assembler = VectorAssembler(inputCols=features_cols, outputCol="features")
24 ventas_df = assembler.transform(ventas_df)
25
26 # Seleccionar columnas finales para el modelo
27 final_df = ventas_df.select("features", "categoría_index")
28
29 # Dividir en entrenamiento y prueba
30 train_df, test_df = final_df.randomSplit([0.8, 0.2], seed=42)
31
32 # Entrenar modelo Random Forest
33 rf = RandomForestClassifier(labelCol="categoría_index", featuresCol="features", numTrees=10)
34 model = rf.fit(train_df)
35
36 # Predecir en test
37 predictions = model.transform(test_df)
38 predictions.select("features", "categoría_index", "prediction").show(10)
39
40 # Evaluar el modelo
41 evaluator = MulticlassClassificationEvaluator(
42     labelCol="categoría_index", predictionCol="prediction", metricName="accuracy")
43 accuracy = evaluator.evaluate(predictions)
44 print(f"\n✓ Precisión del modelo: {accuracy:.2%}")
```

```
+-----+-----+-----+-----+-----+-----+
|id_venta|fecha|cliente|producto|categoría|cantidad|precio_unitario|
+-----+-----+-----+-----+-----+-----+
|1|2025-06-03 04:04:08|Sara Jacobs|now|Hogar|5|8.77|
|2|2025-02-23 05:00:49|Karen Wright|shake|Ropa|5|189.44|
```

3	2025-03-12 09:33:42	Elizabeth Warner	little	Electrónica	2	52.83
4	2025-05-04 07:43:57	Emily Smith	that	Hogar	5	335.19
5	2025-01-17 00:05:13	David Wood	certainly	Hogar	1	160.1

only showing top 5 rows

root

```
-- id_venta: integer (nullable = true)
-- fecha: timestamp (nullable = true)
-- cliente: string (nullable = true)
-- producto: string (nullable = true)
-- categoria: string (nullable = true)
-- cantidad: integer (nullable = true)
-- precio_unitario: double (nullable = true)
```

features	categoria_index	prediction
[1.0,55.14]	0.0	0.0
[1.0,91.93]	3.0	0.0
[1.0,154.49]	2.0	0.0
[1.0,196.84]	0.0	1.0
[1.0,357.08]	2.0	3.0
[2.0,9.58]	2.0	2.0
[2.0,88.05]	0.0	1.0
[2.0,320.79]	0.0	3.0
[2.0,460.4]	2.0	0.0
[2.0,495.84]	1.0	0.0

only showing top 10 rows

✅ Precisión del modelo: 23.53%

Informe de Desarrollo del Proyecto Big Data con Apache Spark

Objetivo General

El objetivo principal fue implementar un **pipeline completo de procesamiento Big Data** utilizando Apache Spark que integre:

- Lectura de datos estructurados y no estructurados desde múltiples fuentes.
- Procesamiento batch y en streaming.
- Consultas SQL optimizadas.
- Entrenamiento de un modelo supervisado con MLlib.
- Documentación técnica y modularización del proyecto para su potencial uso en entornos reales.

Fase 1: Análisis del caso (Lección 1)

La empresa ficticia enfrenta problemas para procesar grandes volúmenes de datos desde distintas fuentes: ventas (estructurado), sensores IoT (semi-estructurado) y logs/redes sociales (no estructurados).

5Vs del Big Data aplicadas

- **Volumen:** gran cantidad de registros heterogéneos.
- **Velocidad:** necesidad de procesamiento en tiempo real (streaming).
- **Variedad:** múltiples formatos (CSV, JSON, texto).
- **Veracidad:** datos ruidosos en logs o sensores.
- **Valor:** los insights impulsan decisiones comerciales.

Tecnologías elegidas

- **Apache Spark** (Core, SQL, Streaming, MLlib).
- Google Colab como entorno de pruebas.
- Archivos planos simulados como entrada (`ventas.csv`, `logs.txt`, `sensores.json`).

Fase 2: Diseño de arquitectura (Lección 2)

Se adoptó una arquitectura basada en **Apache Spark distribuido**:

- **Driver:** coordina la ejecución.
- **Executors:** procesan los datos en paralelo.
- **Cluster Manager:** simulado en Colab con ejecución local (modo standalone).

Módulos utilizados:

- **Spark Core:** ejecución básica, RDDs, acciones y transformaciones.

- **Spark SQL:** consultas estructuradas.
 - **Spark Streaming:** simulación de eventos en tiempo real.
 - **MLlib:** entrenamiento y evaluación del modelo de predicción.
-

Fase 3: Procesamiento Batch con RDD y DataFrame (Lección 3)

Se trabajó con los archivos subidos:

- **RDDs:** lectura con `sc.textFile`, procesamiento de `logs.txt` con `filter`, `map`, `count`.
- **DataFrames:**
 - Lectura de `ventas.csv` con `spark.read.csv`.
 - Lectura de `sensores.json` con `spark.read.json`.
 - Limpieza y selección de columnas.
 - Conversión de tipos de datos (`cast`).

Acciones utilizadas:

- `count()`, `collect()`, `show()`, `mean()`, `sortBy()`, etc.
-

Fase 4: Consultas SQL con Spark SQL (Lección 4)

Se registraron los DataFrames como vistas temporales y se aplicaron consultas SQL:

Consultas ejecutadas:

- Total de ventas por categoría (`GROUP BY categoria`).
- Registros de sensores con temperatura > 32°C.
- Conteo de eventos por nivel de log (INFO, WARN, ERROR).

También se utilizaron funciones como `orderBy`, `filter`, `selectExpr`.

Fase 5: Procesamiento Streaming (Lección 5)

Para simular la entrada en tiempo real se utilizó:

- `readStream.format("socket")` en `localhost:9999` (en entorno local).
- En Colab, se usó un archivo simulado que alimenta una línea por línea con `readStream`.

Lógica aplicada:

- Conversión de texto en palabras.
 - Asignación de marca de tiempo (`current_timestamp()`).
 - Agregación por ventana de 10 minutos cada 5 minutos (`window`).
 - Uso de watermark de 5 minutos para manejar eventos tardíos.
 - Output por consola (`writeStream.format("console")`).
-

Fase 6: Machine Learning Escalable con MLlib (Lección 6)

Se entrenó un modelo de clasificación con Random Forest utilizando las variables `cantidad` y `precio_unitario` como features.

Pasos ejecutados:

1. **Indexado de variables** (`StringIndexer`) para la variable objetivo.
 2. **Ensamblado de features** con `VectorAssembler`.
 3. **Entrenamiento del modelo** con `RandomForestClassifier`.
 4. **Evaluación** con `accuracy` → el modelo alcanzó una precisión de **23.53%** (baja, pero válida como ejercicio).
 5. **Guardado del modelo** con `.write().overwrite().save()` en carpeta `modelo_entrenado`.
-

Conclusión

Este proyecto integra de forma progresiva los conceptos de Big Data utilizando Apache Spark. Desde la lectura distribuida de archivos, pasando por consultas optimizadas, hasta procesamiento en tiempo real y entrenamiento de modelos escalables, se cubren los principales pilares del procesamiento distribuido moderno.

El enfoque modular permite escalar el pipeline y reutilizar componentes en entornos productivos reales.

Archivos generados

Nombre	Descripción
notebook_01_batch.ipynb	Lectura batch con RDDs y DataFrames
notebook_02_sql.ipynb	Consultas SQL con Spark SQL
notebook_03_streaming.ipynb	Procesamiento en tiempo real (streaming)
notebook_04_ml.ipynb	ML escalable con MLlib
modelo_entrenado/	Carpeta de modelo guardado con Random Forest
ventas.csv	Datos estructurados (ventas simuladas)
sensores.json	Datos IoT simulados
logs.txt	Logs de aplicación
README.md	Documentación técnica

Recomendaciones futuras

- Aumentar volumen real de datos para mejorar entrenamiento del modelo.
- Integrar fuentes en tiempo real como Kafka o MQTT.
- Usar Spark en clúster real (AWS EMR, Databricks).
- Validar hiperparámetros del modelo con `CrossValidator`.

Diagrama de Arquitectura del Pipeline Big Data



